

UNACH

Materia: Cultura digital

Semestre: Tercero

Tarea: Actividad autónoma 4

Nombre: Jimmy Erazo Andi

Documentación del Proyecto

Optimización de código para búsqueda de números primos en Python

1. Introducción

El presente proyecto tiene como objetivo analizar y optimizar un programa en Python que busca números primos en un rango de 1 a 100,000.

El código original utiliza una función llamada `es_primo(numero)`, la cual verifica si un número es primo revisando todos los divisores desde 2 hasta `numero - 1`. Aunque este método permite obtener correctamente los números primos, presenta un problema de eficiencia, debido a que realiza una gran cantidad de iteraciones innecesarias.

El principal problema identificado fue el alto tiempo de ejecución. Al analizar el código original con `cProfile`, se evidenció que la función `es_primo()` era la función crítica del programa, ya que fue llamada 100,000 veces y consumió casi todo el tiempo total de ejecución.

Por esta razón, se desarrolló una versión optimizada del código con el propósito de reducir el tiempo de procesamiento y mejorar el rendimiento general del programa.

2. Código original y optimizado

2.1 Código original

```
import time

def es_primo(numero):
    """
    Verifica si un número es primo.
    Esta versión es intencionalmente lenta porque revisa todos los divisores
    desde 2 hasta numero - 1.
    """
    if numero < 2:
        return False

    for divisor in range(2, numero):
        if numero % divisor == 0:
```

```
        return False

    return True

def buscar_primos(limite):
    """
    Busca todos los números primos desde 1 hasta el límite indicado.
    """
    primos = []

    for numero in range(1, limite + 1):
        if es_primo(numero):
            primos.append(numero)

    return primos

# Inicio de medición del tiempo
inicio = time.time()

limite = 100000
primos_encontrados = buscar_primos(limite)

# Fin de medición del tiempo
fin = time.time()

print("Búsqueda de números primos del 1 al", limite)
print("Cantidad de números primos encontrados:", len(primos_encontrados))
print("Primeros 10 números primos:", primos_encontrados[:10])
print("Últimos 10 números primos:", primos_encontrados[-10:])
print("Tiempo de ejecución:", round(fin - inicio, 4), "segundos")
```

2.2 Código optimizado

```
import time
import math
import numpy as np

def es_primo(numero):
    """
    Verifica si un número es primo de forma optimizada.
    Solo revisa divisores hasta la raíz cuadrada del número.
    """
    if numero < 2:
        return False

    if numero == 2:
        return True
```

```
if numero % 2 == 0:
    return False

limite = int(math.sqrt(numero)) + 1

for divisor in range(3, limite, 2):
    if numero % divisor == 0:
        return False

return True

def buscar_primos(limite):
    """
    Busca números primos usando NumPy y list comprehension.
    """
    numeros = np.arange(1, limite + 1)

    primos = [numero for numero in numeros if es_primo(int(numero))]

    return np.array(primos)

inicio = time.time()

limite = 100000
primos_encontrados = buscar_primos(limite)

fin = time.time()

print("Búsqueda optimizada de números primos del 1 al", limite)
print("Cantidad de números primos encontrados:", len(primos_encontrados))
print("Primeros 10 números primos:", primos_encontrados[:10])
print("Últimos 10 números primos:", primos_encontrados[-10:])
print("Tiempo de ejecución:", round(fin - inicio, 4), "segundos")
```

3. Optimización

Para mejorar el rendimiento del código, se aplicaron tres técnicas principales de optimización:

3.1 Reducción del rango del bucle

En el código original, para determinar si un número era primo, se revisaban todos los divisores desde 2 hasta `numero - 1`.

En la versión optimizada, el bucle se redujo para iterar únicamente hasta la raíz cuadrada del número (`sqrt(numero)`). Esta técnica mejora el rendimiento porque si un número tiene un divisor mayor que su raíz cuadrada, necesariamente también tendrá otro divisor menor. Por lo tanto, no es necesario revisar todos los números hasta `numero - 1`.

Además, se agregó una validación para descartar rápidamente los números pares mayores que 2, reduciendo aún más la cantidad de operaciones.

3.2 Uso de list comprehensions

Se utilizó una comprensión de listas para crear la lista de números primos de forma más compacta y eficiente.

Ejemplo aplicado en el código optimizado:

```
primos = [numero for numero in numeros if es_primo(int(numero))]
```

Esta técnica permite reducir la cantidad de líneas de código, mejorar la legibilidad y facilitar la creación de listas a partir de condiciones específicas.

3.3 Uso de NumPy

También se utilizó la biblioteca NumPy para generar el rango de números mediante arrays:

```
numeros = np.arange(1, limite + 1)
```

NumPy permite trabajar con estructuras de datos más eficientes para operaciones numéricas. Aunque en este caso la validación de números primos sigue realizándose mediante una función personalizada, el uso de arrays permite organizar los datos de forma más adecuada para tareas de Ciencia de Datos.

4. Resultados

Para evaluar el rendimiento del código original y del código optimizado, se utilizó la herramienta **cProfile**, la cual permite analizar el tiempo de ejecución de cada función del programa.

4.1 Comparativa de tiempos de ejecución

Versión del código	Tiempo de ejecución
Código original	20.1487 segundos
Código optimizado	0.0768 segundos

Los resultados muestran una mejora significativa en el rendimiento. El código optimizado redujo el tiempo de ejecución de aproximadamente 20.1487 segundos a 0.0768 segundos, según las capturas de pantalla obtenidas desde la consola.

4.2 Capturas de pantalla de ejecución

Código original

```
Búsqueda de números primos del 1 al 100000
Cantidad de números primos encontrados: 9592
Primeros 10 números primos: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
Últimos 10 números primos: [99877, 99881, 99901, 99907, 99923, 99929, 99961, 99971, 99989, 99991]
Tiempo de ejecución: 20.1487 segundos
PS C:\Users\jimmy\OneDrive\Documentos\UNACH\git>
```

Código optimizado

```
Búsqueda optimizada de números primos del 1 al 100000
Cantidad de números primos encontrados: 9592
Primeros 10 números primos: [ 2 3 5 7 11 13 17 19 23 29]
Últimos 10 números primos: [99877 99881 99901 99907 99923 99929 99961 99971 99989 99991]
Tiempo de ejecución: 0.0768 segundos
PS C:\Users\jimmy\OneDrive\Documentos\UNACH\git> |
```

4.3 Análisis de cProfile del código original

En el archivo `profiling_original.txt`, se observó que la función que más tiempo consumió fue:

```
codigo_original.py:5(es_primo)
```

Esta función fue llamada 100,000 veces y consumió aproximadamente 29.099 segundos en la medición de `cProfile`. Esto confirma que el principal problema del código original estaba en la forma en que se verificaba si cada número era primo.

La función `buscar_primos()` también aparece en el análisis, pero su tiempo acumulado depende principalmente de las llamadas repetidas a `es_primo()`.

4.4 Análisis de cProfile del código optimizado

En el archivo `profiling_optimizado.txt`, la función `es_primo()` también fue llamada 100,000 veces, pero su tiempo se redujo aproximadamente a 0.103 segundos.

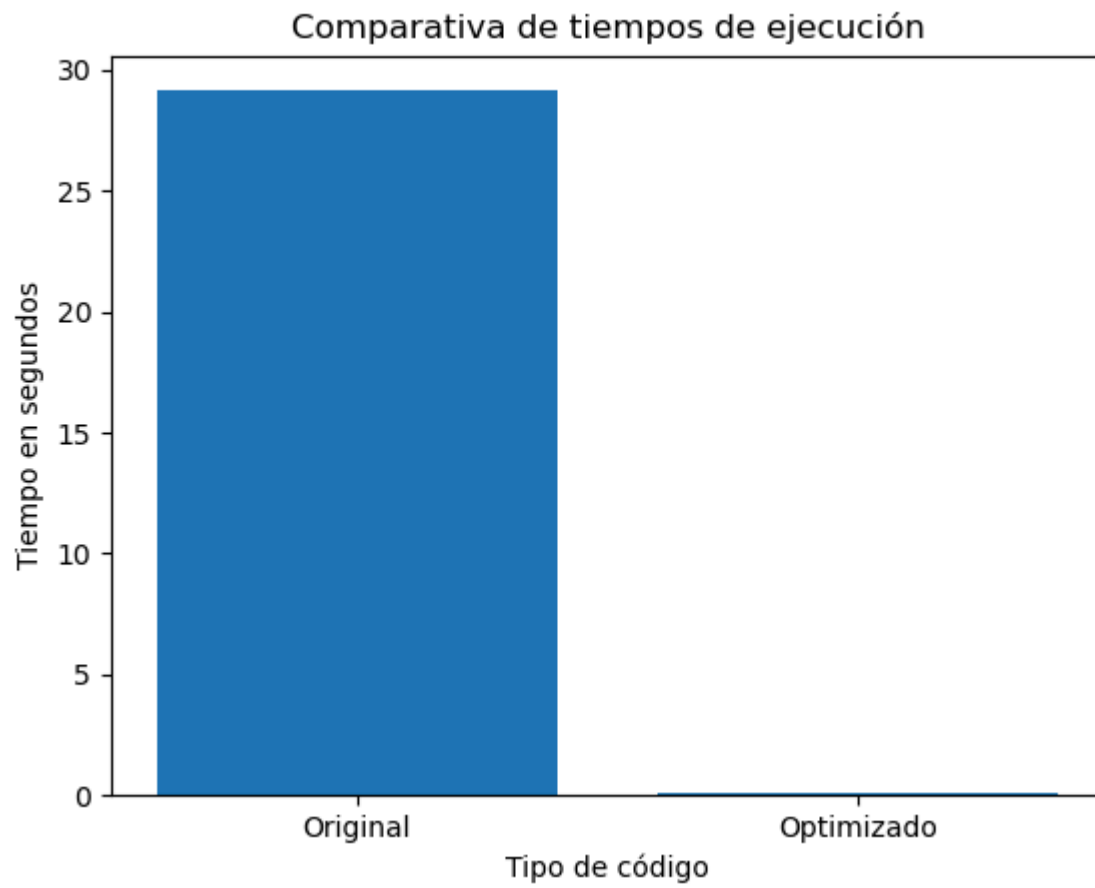
Esto demuestra que la optimización aplicada en la función tuvo un impacto directo en la reducción del tiempo de ejecución.

También se observa que `cProfile` reportó un tiempo total mayor en el código optimizado debido a procesos internos, como la carga de la biblioteca NumPy. Sin embargo, el tiempo medido directamente para la búsqueda de números primos fue de 0.0768 segundos en la consola.

5. Gráficos de resultados

A continuación se presentan los gráficos generados con Matplotlib para visualizar los resultados obtenidos.

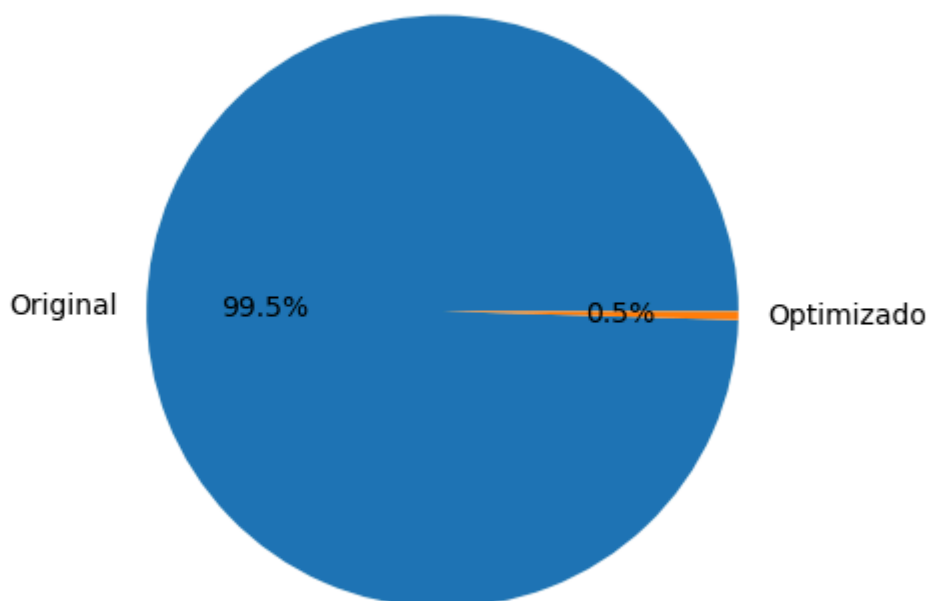
5.1 Comparativa de tiempos de ejecución



Este gráfico permite observar que el código original tarda mucho más que el código optimizado.

5.2 Distribución de tiempos de ejecución

Distribución de tiempos de ejecución



Este gráfico muestra que el código original representa la mayor parte del tiempo total de ejecución, mientras que el código optimizado representa una proporción mínima.

6. Conclusiones

La optimización del código permitió reducir significativamente el tiempo de ejecución del programa de búsqueda de números primos.

El código original funcionaba correctamente, pero era ineficiente porque revisaba todos los posibles divisores de cada número. Esto generaba una gran cantidad de operaciones y aumentaba considerablemente el tiempo de procesamiento.

La versión optimizada mejoró el rendimiento mediante la reducción del rango del bucle, el uso de list comprehensions y la incorporación de NumPy. Como resultado, el programa pasó de tardar aproximadamente 20.1487 segundos a solo 0.0768 segundos en la medición de consola.

El uso de `cProfile` fue fundamental para identificar las funciones críticas y comprobar que la función `es_primo()` era la principal responsable del tiempo de ejecución en ambos casos.

Recomendaciones para futuros desarrollos

- Utilizar herramientas de análisis de rendimiento como `cProfile` antes y después de optimizar un programa.
- Evitar bucles innecesarios o demasiado extensos.
- Aplicar criterios matemáticos para reducir operaciones repetitivas.
- Usar estructuras eficientes como arrays de NumPy cuando se trabaje con datos numéricos.

- Mantener el código documentado para facilitar su comprensión y mantenimiento.
- Comparar siempre los resultados del código original y optimizado para verificar si la mejora fue efectiva.

En conclusión, aplicar buenas prácticas de programación permite desarrollar código más eficiente, claro, mantenible y adecuado para proyectos de Ciencia de Datos.

7. Repositorio en GitHub

Repositorio del proyecto:

<https://github.com/jimmyand1UNACH/Actividad-autonoma-4>